

VANGUARD MILITARY ASSET MANAGEMENT SYSTEM

System Architecture, Technical Specifications & Multi-Base Logistics Guide

UNCLASSIFIED // DEFENSE REQUISITION PROTOTYPE // v2.4.0

1. PROJECT OVERVIEW

across multi-installation theaters of operation. The system coordinates the tracking of Opening Balances, Net Movements (Purchases + Transfers In - Transfers Out), Personnel Assignments, and Operational Expenditures across distributed military bases.

- **Core Accounting Rule:** $\text{Closing Balance} = \text{Opening Balance} + \text{Net Movement (Purchases + In - Out)} - \text{Expended}$.
- **Assumptions:** Multi-base decentralized topology (Fort Liberty, Camp Pendleton, Ramstein Air Base, Naval Station Norfolk). Ledger transactions strictly reconcile balances across customizable date filters.
- **Limitations:** Designed for secure connected intranet networks; edge-to-satellite synchronization protocols for disconnected tactical mobile units represent planned Phase II developments.

telemetry.

- **Frontend (React 18 & Vite):** High-performance Single Page Application built with Vite and styled via Tailwind CSS with a tactical defense theme. Responsive across mobile, tablet, and widescreen command displays. Employs Lucide vector iconography.
- **Database (Relational SQL / better-sqlite3 & PostgreSQL ANSI):** ACID-compliant relational database. Guarantees deterministic ledger calculations, prevents orphaned records via Foreign Key cascades, and provides sub-millisecond query execution with zero external runtime dependencies.

- **bases:** Installations with unique military codes (e.g., BASE-LIBERTY), locations, and commanding officers.
- **equipment_types:** Categorized catalog (Weapons, Vehicles, Ammunition, Communications, Medical Gear).
- **users:** Officer credentials with military rank, assigned base, role clearance, and Bcrypt-hashed credentials.
- **base_inventory:** Initial baseline inventory quantities established prior to transactional date windows.
- **purchases:** Procurement records containing PO number, supplier, unit cost, quantity, total cost, and date.
- **transfers:** Inter-base hardware movements with origin, destination, quantity, tracking code, priority, and status.
- **assignments:** Personnel checkouts recording military service ID, rank, unit, equipment, and check-in return status.
- **expenditures:** Ammunition, combat losses, and decommissioned assets tied to specific operational mission codes.
- **audit_logs:** Immutable append-only security audit log recording every mutation with IP, timestamp, user, and payload.

4. ROLE-BASED ACCESS CONTROL (RBAC)

- **Admin (ADMIN):** Unrestricted command over all four installations, cross-base redeployments, procurement authorization, personnel assignments, operational expenditures, and global security audit trails.
- **Base Commander (BASE_COMMANDER):** Autonomous authority strictly scoped to their assigned base. Can record local purchases, initiate/receive base transfers, check out gear to base personnel, report combat expenditures, and inspect base audit logs.
- **Logistics Officer (LOGISTICS_OFFICER):** Limited logistics role focused on supply chain execution. Authorized to record purchases and initiate inter-base transfers. Strictly restricted from personnel assignments and expenditure approvals (HTTP 403 Forbidden).
- **Enforcement Method:** JWT claims verified via `authenticateToken`; clearance levels enforced via `requireRoles()`; installation data isolation enforced via `checkBaseScope()`.

timestamps, commanding officer ID, role, client IP address, JSON mutation payload, and execution status, establishing complete non-repudiation.

Step 2 (Frontend Setup): Navigate to /frontend, run "npm install", then execute "npm run dev" to initialize and populate the SQLite database.

- **Step 3 (Start Backend):** Run "npm start" inside /backend. The API service will listen on `http://localhost:5001`.
- **Step 4 (Frontend Setup):** Open a second terminal, navigate to /frontend, run "npm install", then "npm run dev". The UI is accessible at `http://localhost:3000`.
- **Step 5 (Production Build):** Execute "npm run build" in /frontend to verify production asset bundling.

- GET /api/dashboard/net-movement-details:** Computes Opening Balance, Closing Balance, Net Movement, Assigned, and Expended assets based on date, base, and equipment filters.
- **GET /api/dashboard/net-movement-details:** [Bonus Pop-up] Returns itemized Purchases, Transfers In, and Transfers Out lists with exact mathematical verification.
 - **GET / POST /api/purchases:** Lists historical procurements; records new asset acquisition with automated audit logging.
 - **GET / POST /api/transfers:** Initiates and tracks inter-base dispatches with live origin base available stock validation.
 - **PATCH /api/transfers/:id/status:** Updates status of in-transit transfers to COMPLETED or CANCELLED.
 - **GET / POST /api/assignments:** Lists and issues equipment to named soldiers with rank and military service number.
 - **PATCH /api/assignments/:id/return:** Marks assigned equipment as returned to the base armory, restoring available stock.
 - **GET / POST /api/expenditures:** Tracks ammunition expended in live-fire exercises or decommissioned gear.
 - **GET /api/audit-logs:** Retrieves queryable audit records filtered by date range, action, domain, and installation.

`{ "id": "commander_0001", "base": "Commander@1234", "role": "BASE_COMMANDER", "name": "Col. Sarah Jenkins (Carson, NC)", "email": "sarah.jenkins@army.mil", "rank": "Colonel", "status": "Active", "last_login": "2024-09-20T14:30:00Z", "permissions": ["purchases", "transfers", "assignments", "expenditures", "audit_logs"] }`

